

IN3370

Ana María Jiménez Güiza

November 2024

Filtrering i bilde- og frekvensdomenet

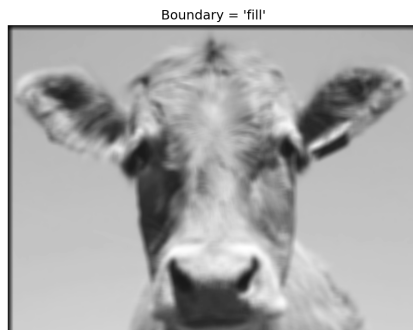
Implementasjon av konvolusjon med riktig kantbehandling

a) Last inn bildet cow.png

```
1  """ a) Upload of image cow.png """
2  filename = "cow.png"
3  image = Image.open(filename).convert('L') # 'L' mode ensures the image is
      ↪ grayscale
4  image = np.array(image)
```

b) Utføring av 15x15 middelveidfiltrering ved bruk av signal.convolve2d

```
1  """ b) 15x15 mean filter using signal.convolve2d """
2
3  # Create a 15x15 mean filter kernel
4  kernel = np.ones((15,15)) / 225
5
6  # Filter with boundary='fill'
7  img_fill = signal.convolve2d(image, kernel, boundary='fill')
8
9  # Filter with boundary='wrap'
10 img_wrap = signal.convolve2d(image, kernel, boundary='wrap')
```

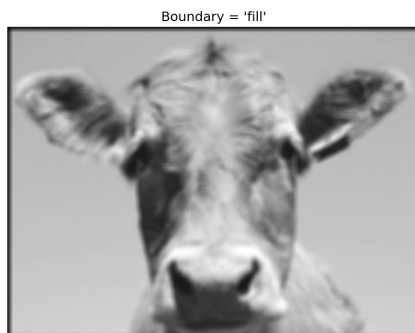


c) Diskuter kort forskjellene i kantene på resultatbildene ved de to løsninger.

Med `boundary='fill'` blir bildets kanter fylt med nuller. Denne metode er nyttig hvis man vil sikre at kantene ikke påvirkes av ombryting, men den kan gi mørke kante på grunn av nullutfylling. Med `boundary='wrap'` blir bildets kanter behandlet som om de er periodiske. Denne metode gir jevnere kanter uten den mørke utfyllingen som man ser i 'fill', ettersom hver kantipiksel har et komplett sett med naboer. På den andre siden, dette kan føre til dårlig kanter hvis bildet ikke har et naturlig repeterende eller kontinuerlig mønster, ettersom nabopikslar fra motsatte kanter kan variere drastisk. Man kan se at toppen av bildet får en veldig mørk kant pga. motsatt kanten har forskjellige mønster.

d) Implementering av FFT-basert filtreringsmetode

```
1  # 'fill' with FFT
2  def fft_fill(image, kernel):
3      pad_shape = (image.shape[0] + kernel.shape[0] - 1, image.shape[1] +
4                  ↪ kernel.shape[1] - 1)
5
6      image_padded = np.pad(image, ((0, pad_shape[0] - image.shape[0]), (0,
7                  ↪ pad_shape[1] - image.shape[1])))
8      kernel_padded = np.pad(kernel, ((0, pad_shape[0] - kernel.shape[0]),
9                  ↪ (0, pad_shape[1] - kernel.shape[1])))
10
11     # Perform FFT on both padded image and kernel
12     image_fft = fft.fft2(image_padded)
13     kernel_fft = fft.fft2(kernel_padded)
14
15     # Multiply in the frequency domain and do the inverse FFT
16     fill_fft = fft.ifft2(image_fft * kernel_fft).real
17
18     return fill_fft
```



```
1  # 'wrap' with FFT
2  def fft_wrap(image, kernel):
```

```

3 padded_kernel = np.zeros(image.shape)
4 padded_kernel[:kernel.shape[0], :kernel.shape[1]] = kernel
5
6 # Perform FFT on both padded image and kernel
7 fft_image = fft.fft2(image)
8 fft_kernel = fft.fft2(padded_kernel)
9
10 # Multiply in the frequency domain and do the inverse FFT
11 wrap_fft = fft.ifft2(fft_image * fft_kernel).real
12
13 return wrap_fft

```



Ytelsesanalyse av romlig og frekvensdomenefiltrering

a) Kjøretiden for forskjellige filterstørrelser

```

1 # Sammenlikning av kjøretiden for forskjellige filterstørrelser
2 kernel_size = [3, 5, 7, 15, 31, 63]
3 for size in kernel_size:
4     kernel = np.ones((size, size)) / size**2
5
6     # Time for convolve2d 'fill'
7     start_time = time.time()
8     img_fill = signal.convolve2d(image, kernel, boundary='fill')
9     end_time = time.time()
10    time_convolve2d_fill = end_time - start_time
11
12
13    # Time for FFT 'fill'
14    start_time = time.time()
15    fill_fft = fft_fill(image, kernel)
16    end_time = time.time()
17    time_fft_fill = end_time - start_time
18
19    print(f"When kernel size is {size}x{size}, time for fill with
    ↳ convolve2d is {time_convolve2d_fill:.5f}, and time with fft is
    ↳ {time_fft_fill:.5f}.")

```

```

20
21     # Time for convolve2d 'wrap'
22     start_time = time.time()
23     img_wrap = signal.convolve2d(image, kernel, boundary='wrap')
24     end_time = time.time()
25     time_convolve2d_wrap = end_time - start_time
26
27     # Time for FFT 'wrap'
28     start_time = time.time()
29     wrap_fft = fft_wrap(image, kernel)
30     end_time = time.time()
31     time_fft_wrap = end_time - start_time
32
33     print(f"When kernel size is {size}x{size}, time for wrap with
    ↪ convolve2d is {time_convolve2d_wrap:.5f}, and time with fft is
    ↪ {time_fft_wrap:.5f}.")

```

Jeg skriver output på et tabell slik at det er mer leslig, og da har vi:

Filterstørrelse	Fill		Wrap	
	Convolve2d	FFT	Convolve2d	FFT
3x3	0.02058	0.048152	0.02080	0.050953
5x5	0.03866	0.104899	0.03951	0.051554
7x7	0.06893	0.074366	0.06893	0.051292
15x15	0.026004	0.078515	0.26040	0.047278
31x31	1.09833	0.058684	1.05204	0.048088
63x63	4.98762	0.099512	5.25997	0.047283

Table 1: Kjøretid i sekunder

Tiden er i sekunder, og vi kan se at FFT er kjappere enn Convolve2d når det kommer til store filter.

b) Gjennomsnittlig kjøretid og standardavvik

```

1  # Gjennomsnittlig kjøretid og standaravvik
2  kernel_size = [3, 5, 7, 15, 31, 63]
3
4  # Data for fill
5  means_convolve2d_fill = []
6  std_devs_convolve2d_fill = []
7  means_fft_fill = []
8  std_devs_fft_fill = []
9
10 # Data for wrap
11 means_convolve2d_wrap = []
12 std_devs_convolve2d_wrap = []
13 means_fft_wrap = []

```

```

14 std_devs_fft_wrap = []
15
16 for size in kernel_size:
17     print(f"Kernel size is {size}x{size}")
18     kernel = np.ones((size, size)) / size**2
19     times_convolve2d_fill = []
20     times_fft_fill = []
21     times_convolve2d_wrap = []
22     times_fft_wrap = []
23     i = 0
24     while i<25:
25         # Time for convolve2d 'fill'
26         start_time = time.time()
27         img_fill = signal.convolve2d(image, kernel, boundary='fill')
28         end_time = time.time()
29         time_convolve2d_fill = end_time - start_time
30         times_convolve2d_fill.append(time_convolve2d_fill)
31
32         # Time for FFT 'fill'
33         start_time = time.time()
34         fill_fft = fft_fill(image, kernel)
35         end_time = time.time()
36         time_fft_fill = end_time - start_time
37         times_fft_fill.append(time_fft_fill)
38
39         # Time for convolve2d 'wrap'
40         start_time = time.time()
41         img_wrap = signal.convolve2d(image, kernel, boundary='wrap')
42         end_time = time.time()
43         time_convolve2d_wrap = end_time - start_time
44         times_convolve2d_wrap.append(time_convolve2d_wrap)
45
46         # Time for FFT 'wrap'
47         start_time = time.time()
48         wrap_fft = fft_wrap(image, kernel)
49         end_time = time.time()
50         time_fft_wrap = end_time - start_time
51         times_fft_wrap.append(time_fft_wrap)
52
53     i += 1
54
55     means_convolve2d_fill.append(np.mean(times_convolve2d_fill))
56     std_devs_convolve2d_fill.append(np.std(times_convolve2d_fill))
57     means_fft_fill.append(np.mean(times_fft_fill))
58     std_devs_fft_fill.append(np.std(times_fft_fill))
59
60     means_convolve2d_wrap.append(np.mean(times_convolve2d_wrap))
61     std_devs_convolve2d_wrap.append(np.std(times_convolve2d_wrap))
62     means_fft_wrap.append(np.mean(times_fft_wrap))
63

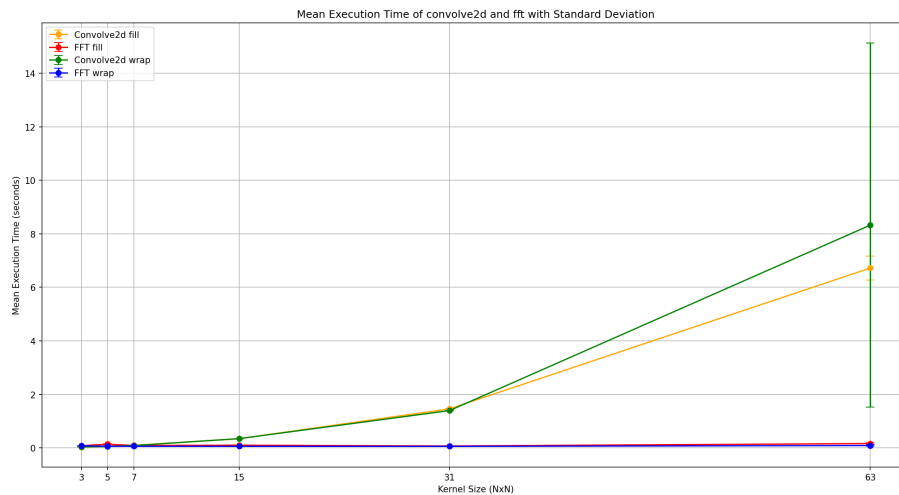
```

```

64     std_devs_fft_wrap.append(np.std(times_fft_wrap))
65
66     print("Trying to plot now")
67     plt.figure(figsize=(10,5))
68     plt.errorbar(kernel_size, means_convolve2d_fill,
69     ↪ yerr=std_devs_convolve2d_fill, fmt='o-', capsize=5,
70         label='Convolve2d fill', color='orange')
71     plt.errorbar(kernel_size, means_fft_fill, yerr=std_devs_fft_fill,
72     ↪ fmt='o-', capsize=5,
73         label='FFT fill', color='red')
74     plt.errorbar(kernel_size, means_convolve2d_wrap,
75     ↪ yerr=std_devs_convolve2d_wrap, fmt='o-', capsize=5,
76         label='Convolve2d wrap', color='green')
77     plt.errorbar(kernel_size, means_fft_wrap, yerr=std_devs_fft_wrap,
78     ↪ fmt='o-', capsize=5,
79         label='FFT wrap', color='blue')
80     plt.xlabel('Kernel Size (NxN)')
81     plt.ylabel('Mean Execution Time (seconds)')
82     plt.title('Mean Execution Time of convolve2d and fft with Standard
83     ↪ Deviation')
84     plt.xticks(kernel_size)
85     plt.grid(True)
86     plt.legend()
87     plt.tight_layout()
88     plt.show()

```

c) Plot av resultatene



d) Hvilke filterstørrelser som favoriserer den ene metoden fremfor den andre

Som vi kan se det er ikke stor forskjell i kjøretiden når filterstørrelse er 3x3, 5x5, eller 7x7, men unntakt for FFT fill når for et eller annen grunn tar det mye lengre tid når filter er 5x5. Dette kunne vi også se i forrige oppgave på tabelen. Jeg har dessverre ikke noe god forklaring for dette.

Når filterne er større ser vi hvordan Convolve2d ikke bare bruker mye mer tid, men også har større standardavikk. Så med filtre som er større enn 15x15 lønner det seg å bruke FFT, spesielt når man bruker 'wrap' siden den har de verste kjøretidmedianen og størst standardavikk når filteret er 63x63.

Ideelt frekvensdomenefilter for lavpassfiltrering

a) Implementering av et "ideelt" lavpassfilter

```
1  # lavpassfilter ved hjelp av sirkelutformet, velg radius slik at likner
   ↪ 15x15 convolve2d
2  n, m = image.shape
3  D0 = 0.1
4  H = np.zeros((n, m))
5
6  for u in range(n):
7      for v in range(m):
8          D = np.sqrt(((u-0.5*n)/(0.5*n))**2+((v-m*0.5)/(m*0.5))**2)
9          if D <= D0:
10             H[u,v] = 1
11
12  F = fft.fft2(image)
13  img_h = (fft.ifft2(F*fft.fftshift(H))).real
```

b) Bildet med 15x15 gjennomsnittfilter og det ideelle frekvensdomenefilteret



c) Forskjeller i resultatene

Forskjellen ligger hovedsakelig i hvordan de to metodene implementerer grenseverdier. Med `convolve2d` med `boundary = 'wrap'`, blir filtreringen lineært. På den andre side, det lavpassfilterert blir filtreringen sirkulær. Dette er på grunn av filteret er definert ved hjelp av en sirkel med radius D_0 som slipper gjennom bare lave frekvenser opp til en bestemt radius og blokkerer høyere frekvenser. På grunn av den skarpe cutoffen ser vi at bildet har ringing-effekt.

`Convolve2d` tar i hensyn lokale piksler for filtreringen mens lavpassfilteret tar hensyn til globale frekvenser. Dette gjør at det ideelle lavpassfilteret er mer globalt og gir en presis lavpass-effekt, men kan føre til ringing effekt. Og `convolve2d` kan håndtere bedre lokale detaljer men kan føre til dårlig kanter.